

PRACTICAS DE CLASE

ASIGNATURA:

INGENIERIA DE SOFTWARE

GRUPO:

A

ESPECIALIDAD:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

DOCENTE:

FIERRO MENDOZA JOSE GUILLERMO

ALUMNA:

SILVA PANIAGUA JENNIFER MARIA CRUZ

NUMERO DE CONTROL:

23031052

Documentación del Desarrollo del Proyecto FULLSTACK – Carrito de Compras

Introducción

El presente proyecto consiste en el desarrollo de una aplicación tipo FULLSTACK enfocada en la gestión de productos y carrito de compras utilizando el stack PERN adaptado a MySQL mediante Sequelize.

El objetivo principal fue construir una aplicación moderna con:

- Frontend en React
- Backend con Express y Node.js
- Base de datos MySQL
- Arquitectura Cliente/Servidor separada
- API REST
- ORM Sequelize
- Uso de CORS
- Logging con Morgan
- Validaciones
- Testing
- Documentación Swagger

Etapas 1 – Desarrollo de la Interfaz Frontend

Para la construcción inicial de la interfaz gráfica se tomaron como referencia dos videos tutoriales enfocados en:

- Diseño responsive
- Implementación de un carrito de compras
- Organización visual de productos
- Estilos modernos utilizando TailwindCSS

Gracias a estos tutoriales se logró desarrollar:

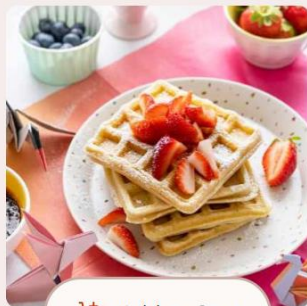
Funcionalidades iniciales

- Visualización de productos
- postres
- Diseño responsive
- Tarjetas dinámicas de productos
- Carrito de compras funcional
- Confirmación de orden
- Manejo visual de cantidades
- Interfaz moderna y adaptable

Interfaz

<http://localhost:5173>

Desserts



 Add to Cart

Waffle

Waffle with Berries

\$6.50



 Add to Cart

Crème Brûlée

Vanilla Bean Crème Brûlée

\$7.00



 Add to Cart

Macaron

Macaron Mix of Five

\$8.00





Your Cart (2)

Red Velvet Cake
1x @\$4.50 \$4.50



Salted Caramel Brownie
1x @\$4.50 \$4.50



Order Total

\$9.00



This is a **carbon neutral** delivery

Confirm Order



Order Confirmed

We hope you enjoy your food!



Red Velvet Cake
1x @\$4.50

\$4.50



Salted Caramel Brownie
1x @\$4.50

\$4.50

Order Total

\$9.00

Start New Order

Tecnologías utilizadas inicialmente

Frontend

- React
- Vite
- TailwindCSS
- React Router DOM

Estructura inicial del proyecto

ACTIVIDADI/

```
|—— public/
|—— src/
|   |—— assets/
|   |—— components/
|   |—— pages/
|   |—— routes/
|   |—— store/
|   |—— App.jsx
|   |—— index.css
|   |—— main.jsx
|—— data.json
|—— package.json
|—— vite.config.js
```

Funcionamiento inicial

Inicialmente los productos eran obtenidos desde un archivo local:

data.json

Los productos contenían:

- Nombre
- Categoría
- Precio
- Imágenes
- Información visual

Ejemplo:

```
{  
  "name": "Waffle with Berries",  
  "category": "Waffle",  
  "price": 6.50  
}
```

En esta etapa el proyecto funcionaba únicamente del lado del cliente (Frontend).

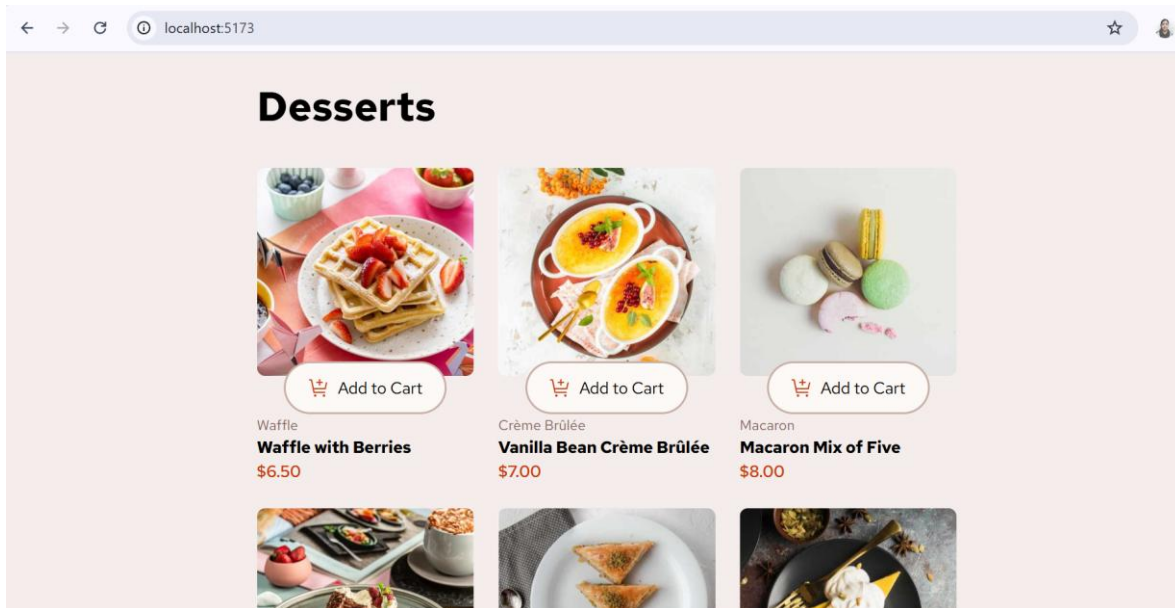
Etapas 2 – Conversión a Arquitectura FULLSTACK

Posteriormente se comenzó la separación entre:

Cliente

Frontend React ejecutándose en:

<http://localhost:5173>



Servidor

Backend Express ejecutándose en:

<http://localhost:3000>



Creación del Backend

Se creó una carpeta independiente para el servidor:

server/

Con el objetivo de implementar:

- API REST
- Conexión a base de datos
- Manejo de peticiones HTTP
- Arquitectura FULLSTACK real

Instalación de dependencias Backend

Dependencias principales

npm install express cors morgan nodemon

Función de cada dependencia

Dependencia	Función
Express	Creación del servidor backend
CORS	Permitir comunicación entre frontend y backend
Morgan	Logging de peticiones HTTP
Nodemon	Reinicio automático del servidor

Configuración inicial del servidor

Se creó:

server.js

Encargado de iniciar el servidor Express.

app.js

Encargado de configurar middlewares y rutas.

Uso de CORS

Se implementó:

app.use(cors())

¿Qué es CORS?

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad que permite la comunicación entre aplicaciones ejecutándose en diferentes puertos o dominios.

En este caso:



- React corre en puerto 5173
- Express corre en puerto 3000

Sin CORS el navegador bloquearía las peticiones entre ambos.



Uso de Morgan

Se implementó:

```
app.use(morgan('dev'))
```

Función

Morgan permite visualizar en consola:

- Métodos HTTP
- Tiempo de respuesta
- Estado de las peticiones

Ejemplo:

```
GET /api/products 200
```

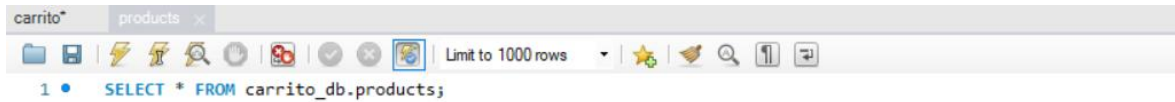
Etapas 3 – Integración con MySQL

Se decidió utilizar MySQL como sistema gestor de base de datos.

Creación de la Base de Datos

Se creó la base:

CREATE DATABASE carrito_db;



id	name	price	stock	image	createdAt	updatedAt	category
1	Waffle with Berries	6.5	10	image-waffle-desktop.jpg	2026-05-21 00:51:02	2026-05-21 00:51:02	Waffle
2	Vanilla Bean Crème Brûlée	7	10	image-creme-brulee-desktop.jpg	2026-05-21 01:06:50	2026-05-21 01:06:50	Crème Brûlée
3	Macaron Mix of Five	8	10	image-macaron-desktop.jpg	2026-05-21 01:07:34	2026-05-21 01:07:34	Macaron
4	Classic Tiramisu	5.5	10	image-tiramisu-desktop.jpg	2026-05-21 01:07:48	2026-05-21 01:07:48	Tiramisu
5	Pistachio Baklava	4	10	image-baklava-desktop.jpg	2026-05-21 01:07:59	2026-05-21 01:07:59	Baklava
6	Lemon Meringue Pie	5	10	image-meringue-desktop.jpg	2026-05-21 01:08:21	2026-05-21 01:08:21	Pie
7	Red Velvet Cake	4.5	10	image-cake-desktop.jpg	2026-05-21 01:08:34	2026-05-21 01:08:34	Cake
8	Salted Caramel Brownie	4.5	10	image-brownie-desktop.jpg	2026-05-21 01:09:15	2026-05-21 01:09:15	Brownie
9	Vanilla Panna Cotta	6.5	10	image-panna-cotta-desktop.jpg	2026-05-21 01:09:34	2026-05-21 01:09:34	Panna Cotta
10	Test Product	100	5	test.jpg	2026-05-25 02:11:35	2026-05-25 02:11:35	Test

Instalación de Sequelize

Se instalaron:

npm install sequelize mysql2

Función

Herramienta Función

Sequelize	ORM para trabajar con MySQL
mysql2	Driver de conexión MySQL

Configuración de Sequelize

Se creó:

server/src/config/Database.js

Con la configuración:

```
const { Sequelize } = require('sequelize')
```

```
const sequelize = new Sequelize(  
  'carrito_db',  
  'jenni',  
  '1234',  
  {  
    host: 'localhost',  
    dialect: 'mysql'  
  }  
)
```

```
module.exports = sequelize
```

```
Executing (default): SHOW INDEX FROM `Products`  
Tablas sincronizadas  
Servidor corriendo en puerto 3000  
5:11 PM (UTC-6) SELECT * FROM `Products`
```

Problemas encontrados durante la conexión

Durante la conexión surgieron distintos errores:

Errores enfrentados

- Sequelize no instalado
- Ruta incorrecta del archivo Database.js
- Error de permisos MySQL
- Usuario sin acceso a la base de datos

Solución implementada

Se creó un nuevo usuario MySQL:

```
CREATE USER 'jenni'@'localhost' IDENTIFIED BY '1234';
```

Posteriormente se otorgaron permisos:

```
GRANT ALL PRIVILEGES ON carrito_db.* TO 'jenni'@'localhost';  
FLUSH PRIVILEGES;
```

Resultado

Se logró establecer correctamente la conexión entre:

- Node.js
- Express
- Sequelize
- MySQL

Mensaje obtenido:

Base de datos conectada

Servidor corriendo en puerto 3000

Etapa 4 – Creación del Modelo Product

Se creó el modelo:

Product.js

Utilizando Sequelize.

Campos definidos

Campo	Tipo
id	INTEGER
name	STRING
category	STRING
price	FLOAT
image	STRING



Campo	Tipo
stock	INTEGER

Sincronización de tablas

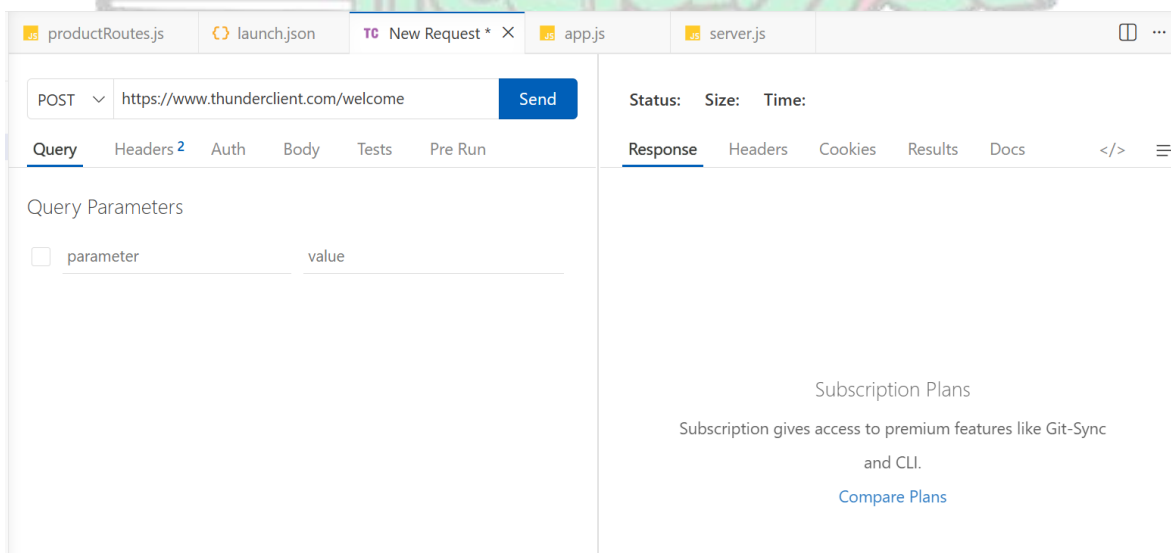
Se utilizó:

```
sequelize.sync({ alter: true })
```

Con esto Sequelize creó automáticamente la tabla:

Products

Dentro de MySQL.



productRoutes.js launch.json TC New Request * X app.js server.js

POST https://www.thunderclient.com/welcome Send

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter value

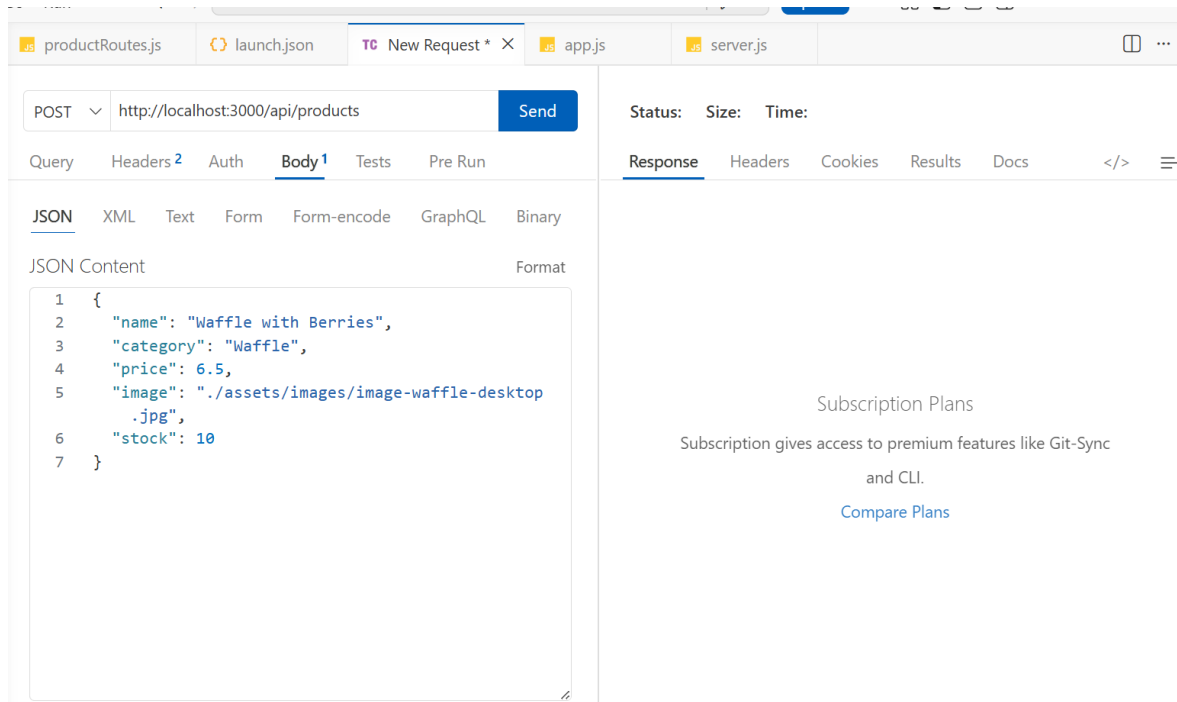
Status: Size: Time:

Response Headers Cookies Results Docs </> ≡

Subscription Plans

Subscription gives access to premium features like Git-Sync and CLI.

[Compare Plans](#)



Etapa 5 – Construcción de la API REST

Se desarrollaron:

Controllers

productController.js

Routes

productRoutes.js

Endpoints creados

Método	Endpoint	Función
GET	/api/products	Obtener productos
POST	/api/products	Crear productos

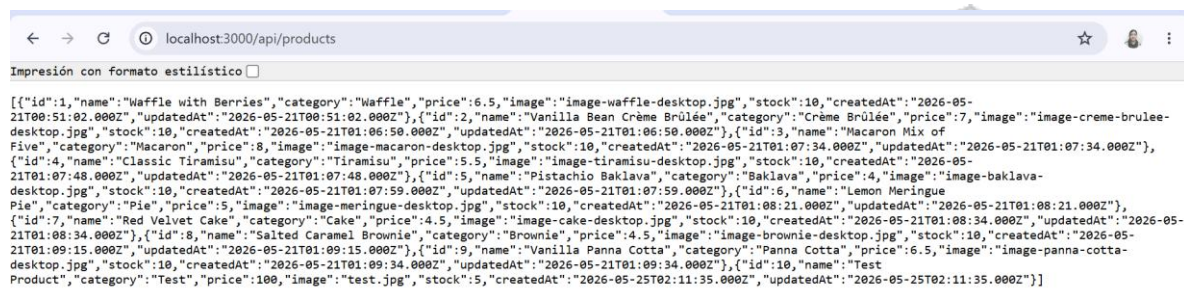
Pruebas iniciales



Se realizaron pruebas mediante navegador y Thunder Client.

Ejemplo:

<http://localhost:3000/api/products>



Posteriormente se comenzaron a insertar productos reales en MySQL.

Estado actual del proyecto

Actualmente el proyecto cuenta con:

Frontend

- Interfaz completa
- Carrito funcional
- Diseño responsive
- TailwindCSS

Backend

- Express
- API REST
- Sequelize
- MySQL
- CORS
- Morgan

Base de datos



- Tabla Products funcional
- Conexión establecida correctamente

Próximas etapas

Las siguientes implementaciones pendientes son:

- Integrar React con la API REST
- Eliminar dependencia de data.json
- Implementar Swagger
- Implementar Jest
- Implementar Supertest
- Validaciones con Express Validator
- Validaciones con Zod
- CRUD completo
- Testing de endpoints

Etapa 6 – Integración del Frontend con la API REST

Una vez funcional el backend y la base de datos, se inició el proceso de integración entre React y Express para convertir completamente la aplicación en una arquitectura FULLSTACK real.

Problema inicial

Inicialmente los productos eran obtenidos desde:

data.json

Esto implicaba que:

- Los productos solo existían del lado del frontend
- No existía persistencia real
- Los datos no provenían de la base de datos MySQL

Objetivo de esta etapa

Se buscó:

- Eliminar dependencia de data.json
- Consumir productos desde la API REST
- Obtener productos directamente desde MySQL
- Mantener intacta la interfaz gráfica ya desarrollada

Instalación de Axios

Para realizar peticiones HTTP desde React se instaló:

`npm install axios`

Función de Axios

Axios permite:

- Realizar peticiones GET
- Realizar peticiones POST
- Consumir APIs REST fácilmente
- Conectar frontend con backend

Uso de Zustand

El proyecto ya utilizaba Zustand para el manejo global del carrito de compras.

El store originalmente contenía:

`products: data`

Donde data provenía de data.json.

Conversión del Store a consumo de API

Se modificó el store:

`src/store/cart.store.js`

Para reemplazar el arreglo local por peticiones HTTP hacia Express.

Implementación de `getProducts()`

Se agregó una función asíncrona:

```
getProducts: async () => {  
  
  const response = await axios.get(  
    'http://localhost:3000/api/products'  
  )  
  
  set({  
    products: response.data  
  })  
  
}
```

Función

Esta función:

- Consume la API REST
- Obtiene productos desde MySQL
- Actualiza automáticamente el estado global

Modificación de `CardsContainer`

Se modificó:

`CardsContainer.jsx`

Para cargar productos automáticamente al iniciar la aplicación.

Se implementó:

```
useEffect(() => {  
  getProducts()
```

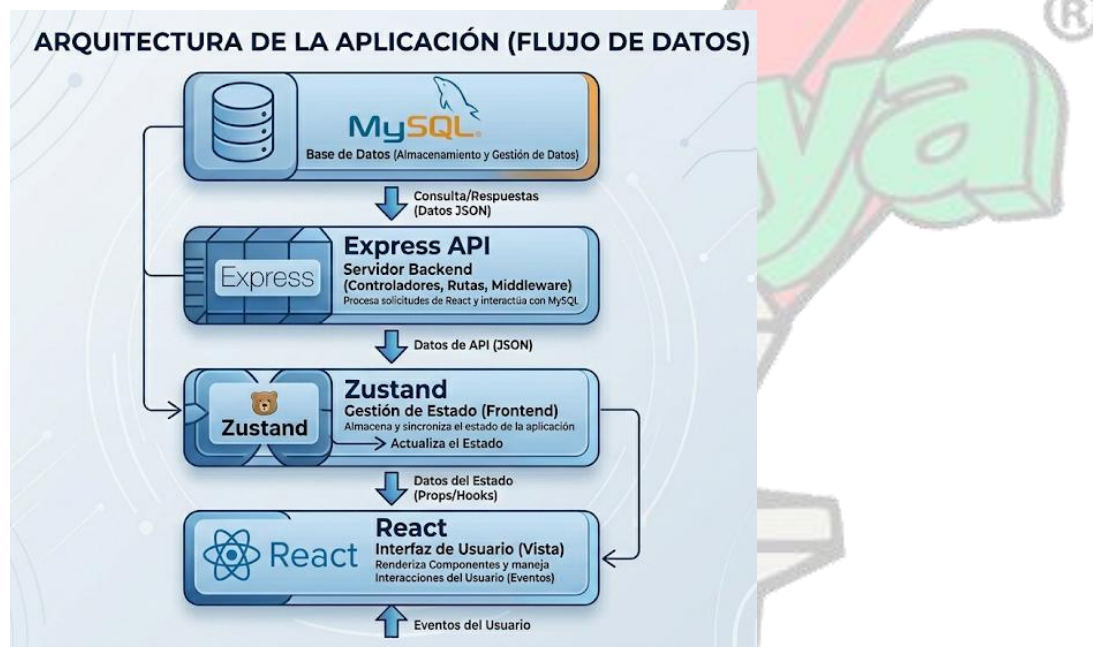
```
} , [])
```

Resultado obtenido

La aplicación comenzó a consumir productos reales desde:

`http://localhost:3000/api/products`

Logrando el flujo:



Eta 7 – Inserción de Productos Reales

Se comenzaron a insertar productos mediante Thunder Client.

Uso de Thunder Client

Thunder Client fue utilizado para probar endpoints HTTP directamente desde VS Code.

Endpoint utilizado

POST `/api/products`

URL:

`http://localhost:3000/api/products`

Ejemplo de inserción

```
{  
  "name": "Waffle with Berries",  
  "category": "Waffle",  
  "price": 6.5,  
  "image": "image-waffle-desktop.jpg",  
  "stock": 10  
}
```

Resultado

Los productos comenzaron a almacenarse correctamente dentro de MySQL y posteriormente eran mostrados automáticamente en React.

Etaapa 8 – Manejo de Imágenes Dinámicas

Durante la integración surgió un problema relacionado con las imágenes.

Problema encontrado

Las imágenes inicialmente estaban definidas así:

`./assets/images/image-waffle-desktop.jpg`

Esto funcionaba únicamente desde el frontend local, pero no desde datos obtenidos por API.

Problema técnico

React y Vite no podían resolver correctamente rutas dinámicas provenientes desde MySQL.

Solución implementada

Se decidió mover las imágenes hacia:
public/assets/images

Nueva estructura

public/

└── assets/

└── images/

Cambio realizado en ProductCard

Se reemplazó:

```
src={image.mobile}
```

Por:

```
src={` /assets/images/${image}` }
```

Resultado

Las imágenes comenzaron a renderizarse correctamente utilizando:
<http://localhost:5173/assets/images/>

Etapas 9 – Conversión del Modelo de Imágenes

Inicialmente el frontend utilizaba objetos complejos para imágenes:

```
image: {  
  mobile,  
  tablet,  
  desktop  
}
```

Problema

La base de datos almacenaba únicamente strings simples.

Solución

Se simplificó el modelo para utilizar únicamente:

image-waffle-desktop.jpg

Como referencia de imagen.

Beneficios obtenidos

- Menor complejidad
- Compatibilidad con MySQL
- Mayor facilidad de renderizado
- Menor cantidad de datos almacenados

Etapa 10 – Implementación de React Router DOM

Para cumplir con los requisitos del proyecto se implementó:

```
npm install react-router-dom
```

Objetivo

Agregar navegación profesional mediante rutas dinámicas.

Estructura creada

src/

├── pages/

├── routes/



Página Home

Se creó:

Home.jsx

Conteniendo:

- Productos
- Carrito
- Confirmación de orden

Creación del Router

Se creó:

AppRouter.jsx

Utilizando:

BrowserRouter

Routes

Route

Resultado

La aplicación comenzó a utilizar navegación basada en rutas:

/

Preparando la aplicación para futuras vistas como:

- Admin
- Login
- Historial de órdenes
- Gestión de productos

Estado actual del proyecto

Actualmente el sistema cuenta con:

Frontend

- React
- TailwindCSS
- Zustand
- Axios
- React Router DOM

Backend

- Node.js
- Express
- Sequelize
- Morgan
- CORS

Base de datos

- MySQL
- Tabla Product funcional

Arquitectura

- Cliente y servidor separados
- API REST funcional
- Persistencia real en MySQL

Funcionalidades logradas

Productos dinámicos desde MySQL
Carrito funcional
API REST conectada
Frontend conectado al backend

Inserción dinámica de productos
Manejo de imágenes dinámicas
Arquitectura FULLSTACK real
Navegación con React Router DOM
Estado global con Zustand
Persistencia real de datos

Etapas 11 – Implementación de Swagger

Con el objetivo de documentar la API REST se implementó Swagger.®

Instalación

Se instalaron las dependencias:

```
npm install swagger-ui-express swagger-jsdoc
```

Objetivo

Swagger permite:

- Documentar endpoints
- Probar peticiones HTTP
- Visualizar rutas de la API
- Facilitar pruebas y mantenimiento

Configuración

Se creó:

```
server/src/docs/swagger.js
```

Conteniendo la configuración OpenAPI.

Posteriormente se agregó el middleware:

```
app.use(  
  '/api/docs',  
  swaggerUi.serve,  
  swaggerUi.setup(swaggerSpec)  
)
```



Documentación de endpoints

Se documentaron:

Método Endpoint

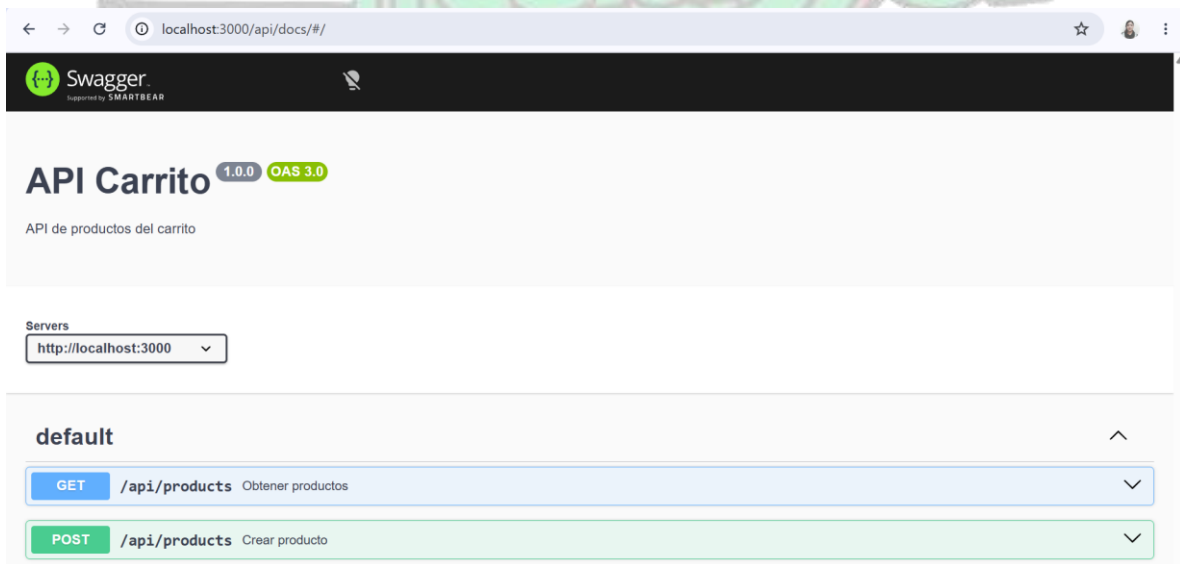
GET	/api/products
POST	/api/products

Resultado

La documentación quedó disponible en:

<http://localhost:3000/api/docs>

Permitiendo ejecutar endpoints directamente desde el navegador.



Etaapa 12 – Implementación de Express Validator

Para validar entradas de datos en el backend se implementó Express Validator.

Instalación

```
npm install express-validator
```

Objetivo

Validar:

- Campos vacíos

- Tipos de datos
- Entradas incorrectas

Validaciones implementadas

Se validaron:

- name
- price
- stock

Ejemplo:

```
body('name')  
  .notEmpty()  
  .withMessage('El nombre es obligatorio')
```

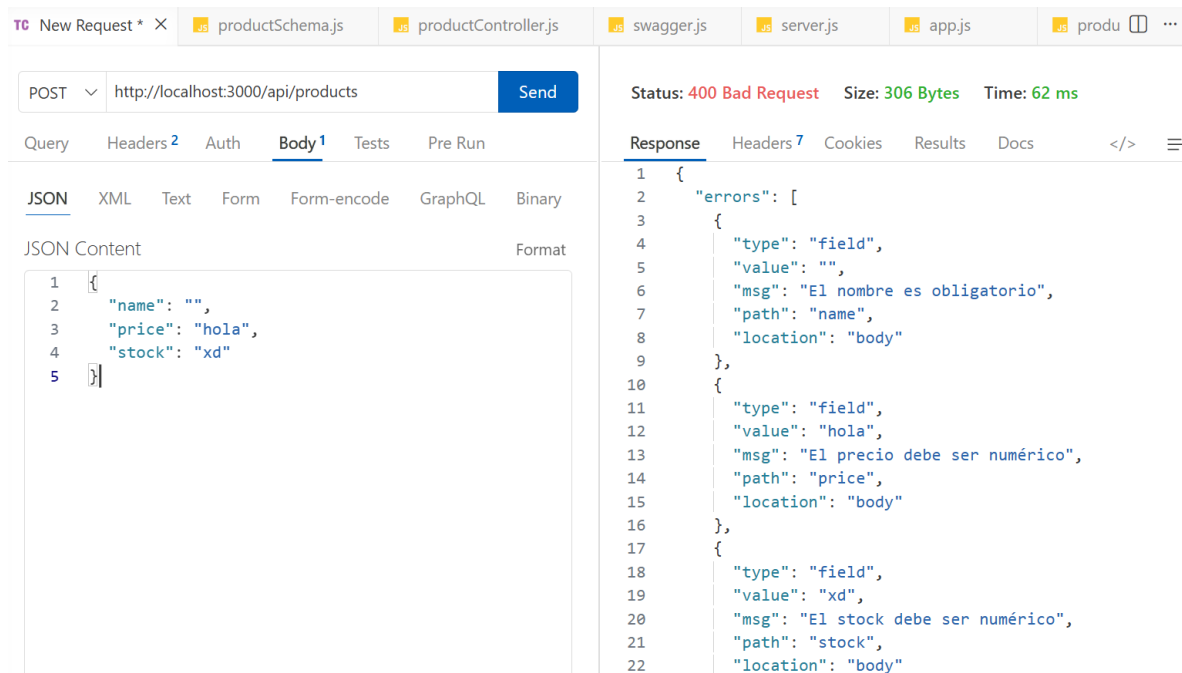
Resultado

El backend ahora evita insertar productos inválidos en MySQL.

En Thunder Client:

manda:

```
{  
  "name": "",  
  "price": "hola",  
  "stock": "xd"  
}
```



The screenshot shows a REST client interface with a POST request to `http://localhost:3000/api/products`. The request body is a JSON object: `{ "name": "", "price": "hola", "stock": "xd" }`. The response is a 400 Bad Request with a status of 400, size of 306 Bytes, and time of 62 ms. The response body is a JSON array of error objects:

```
1 {
2   "errors": [
3     {
4       "type": "field",
5       "value": "",
6       "msg": "El nombre es obligatorio",
7       "path": "name",
8       "location": "body"
9     },
10    {
11      "type": "field",
12      "value": "hola",
13      "msg": "El precio debe ser numérico",
14      "path": "price",
15      "location": "body"
16    },
17    {
18      "type": "field",
19      "value": "xd",
20      "msg": "El stock debe ser numérico",
21      "path": "stock",
22      "location": "body"
23    }
24  ]
25 }
```

Etaapa 13 – Implementación de Zod

Además de Express Validator se implementó Zod para realizar validaciones mediante esquemas.

Instalación

`npm install zod`

Objetivo

Validar estructuras completas de objetos utilizando schemas.

Implementación

Se creó:

`server/src/schemas/productSchema.js`

Ejemplo:

```
const productSchema = z.object({
  name: z.string(),
  price: z.number(),
  stock: z.number()
})
```



```
stock: z.number()  
})
```

Beneficios

- Validaciones más robustas
- Validaciones tipadas
- Mayor seguridad de datos

```
const { z } = require('zod')  
  
const productSchema = z.object({  
  
  name: z  
    .string()  
    .min(1, 'El nombre es obligatorio'),  
  
  category: z  
    .string()  
    .min(1, 'La categoría es obligatoria'),  
  
  price: z  
    .number({  
      invalid_type_error: 'El precio debe ser número'  
    }),  
  
  stock: z  
    .number({  
      invalid_type_error: 'El stock debe ser número'  
    }),  
  
  image: z  
    .string()  
    .min(1, 'La imagen es obligatoria')  
  
})  
  
module.exports = productSchema
```

Etapa 14 – Testing con Jest y Supertest

Para realizar pruebas automatizadas se implementaron Jest y Supertest.

Instalación

```
npm install --save-dev jest supertest
```

Objetivo

Verificar automáticamente:

- Respuestas del servidor
- Endpoints
- Códigos HTTP

Tests implementados

Se realizaron pruebas sobre:

Método Endpoint

GET /api/products

POST /api/products

Ejemplo

```
expect(response.statusCode).toBe(200)
```

Resultado

Se logró automatizar pruebas del backend correctamente.



```
const request = require('supertest')

const app = require('../src/app')

describe('Rutas de productos', () => {

  test('GET /api/products debe responder 200', async () => {

    const response = await request(app)
      .get('/api/products')

    expect(response.statusCode).toBe(200)

  })

  test('POST /api/products debe crear producto', async () => {

    const newProduct = {

      name: 'Test Product',
      category: 'Test',
      price: 100,
      stock: 5,
      image: 'test.jpg'

    }

    const response = await request(app)
      .post('/api/products')
      .send(newProduct)

    expect(response.statusCode).toBe(201)

  })

})
```

5. AGREGAR

Arquitectura Final del Proyecto

El flujo final de la aplicación quedó de la siguiente manera:

React Frontend

↓

Axios

↓

Express API REST



Sequelize ORM



MySQL Database

Flujo completo

1. React realiza petición HTTP mediante Axios.
2. Express recibe la petición.
3. Sequelize ejecuta consultas SQL.
4. MySQL devuelve los datos.
5. Express responde en formato JSON.
6. React renderiza productos dinámicamente.

Conclusiones

Durante el desarrollo del proyecto se logró implementar una arquitectura FULLSTACK funcional utilizando React, Express, Sequelize y MySQL.

Se consiguió:

- Separar frontend y backend
- Implementar una API REST
- Conectar React con MySQL
- Validar datos
- Documentar endpoints
- Realizar testing automatizado

Además, el proyecto permitió comprender el funcionamiento completo de una aplicación moderna basada en cliente-servidor.

Tecnologías Finales Utilizadas

Tecnología **Uso**

React	Frontend
TailwindCSS	Estilos
Zustand	Estado global
Axios	Peticiones HTTP
Express	Backend
Sequelize	ORM
MySQL	Base de datos
Swagger	Documentación
Jest	Testing
Supertest	Testing API
Zod	Validaciones
Express Validator	Validaciones
Morgan	Logging
CORS	Comunicación cliente-servidor

Componente **¿Qué es?**

¿Para qué sirve?

**Tecnologías
usadas**

Frontend	Es la parte visual que ve el usuario	Mostrar productos, carrito y pantallas de la aplicación	React, TailwindCSS, React Router DOM
Backend	Es la lógica del servidor	Procesar peticiones, manejar datos y conectar con la base de datos	Node.js, Express

Componente	¿Qué es?	¿Para qué sirve?	Tecnologías usadas
Base de datos	Lugar donde se almacenan los datos	Guardar productos, precios, stock e información permanente	MySQL
API REST	Sistema de comunicación entre frontend y backend	Permite enviar y recibir datos mediante URLs	Express®
Sequelize	ORM para MySQL	Facilita consultas a la base de datos usando JavaScript	Sequelize
React Router DOM	Sistema de rutas	Permite navegación entre páginas sin recargar	React Router DOM
Zustand	Manejo global de estado	Controlar carrito y productos desde cualquier componente	Zustand
Axios	Cliente HTTP	Consumir la API REST desde React	Axios
CORS	Middleware de seguridad	Permite comunicación entre frontend y backend en distintos puertos	cors
Morgan	Logger HTTP	Mostrar en consola las peticiones realizadas al servidor	morgan
Swagger	Documentación visual de API	Mostrar y probar endpoints del backend	swagger-ui-express
Jest	Framework de testing	Crear pruebas automáticas del sistema	Jest

Componente	¿Qué es?	¿Para qué sirve?	Tecnologías usadas
Supertest	Testing de endpoints	Probar rutas y respuestas HTTP del backend	Supertest
Express Validator	Validación de formularios	Verificar que los datos enviados sean correctos	express-validator
Zod	Validación avanzada	Validar estructuras y tipos de datos	Zod
Thunder Client	Cliente de pruebas HTTP	Probar endpoints directamente desde VS Thunder Client Code	
Vercel	Plataforma de deploy frontend	Publicar la aplicación React en internet	Vercel
Render	Plataforma de deploy backend	Publicar el servidor Express en internet	Render